Node.js

Node.js is a JavaScript runtime built on Chrome's V8 engine. It allows running JavaScript outside the browser (e.g., backend, servers, scripts). Think of Node.js as the engine that powers JavaScript on your computer.

npm vs npx

npm (Node Package Manager) is used to install and manage libraries. npx (Node Package Execute) runs packages directly without installing them permanently.

```
# Example using npm
npm install react

# Example using npx
npx create-react-app myApp
```

Express.js

Express.js is a web framework for Node.js that simplifies server and API creation.

```
// Without Express
const http = require("http");
const server = http.createServer((req, res) => {
  if (req.url === "/") res.end("Home Page");
  else if (req.url === "/about") res.end("About Page");
});
server.listen(3000);

// With Express
const express = require("express");
const app = express();
app.get("/", (req, res) => res.send("Home Page"));
app.get("/about", (req, res) => res.send("About Page"));
app.listen(3000, () => console.log("Server running..."));
```

SQL vs NoSQL

SQL (Relational DB): Structured, fixed schema, tables. Best for banking/e-commerce. NoSQL: Flexible schema, JSON-like, designed for scalability. Best for modern apps.

MongoDB

MongoDB is a document-based NoSQL database storing data as JSON-like documents.

```
// Example MongoDB document
{
  "id": 1,
  "name": "John",
  "age": 25,
  "skills": ["React", "Node.js"]
}

// Query
db.users.find({ age: { $gt: 25 } });
```

Initialize Node Project

Use npm init to create package.json and manage dependencies.

```
mkdir my-app && cd my-app
npm init -y
npm install express
```

Nodemon

Nodemon automatically restarts Node.js applications when file changes are detected.

```
npm install --save-dev nodemon

// package.json scripts
"scripts": {
  "start": "node index.js",
  "dev": "nodemon index.js"
}

# Run
npm run dev
```

Modules in Node.js

Types: 1. Core (built-in like fs, http, path) 2. Local (your own files) 3. Third-party (npm installed)

```
// Core Module
const fs = require("fs");
fs.writeFileSync("hello.txt", "Hello");

// Local Module
// math.js
module.exports = (a, b) => a + b;

// app.js
const add = require("./math");

// Third-party Module
const express = require("express");
```

CommonJS vs ES Modules

CommonJS uses require/module.exports, synchronous. ES Modules use import/export, asynchronous.

```
// CommonJS
const add = require("./math");
module.exports = fn;

// ESM
import { add } from "./math.js";
export default fn;
```

Named vs Default Exports (ESM)

Named exports allow multiple exports with fixed names. Default export allows only one main export with flexible name.

```
// Named
export const add = (a,b)=>a+b;
export const sub = (a,b)=>a-b;

// Default
export default function multiply(a,b){ return a*b; }

// Import
import multiply, { add, sub } from "./math.js";
```

Named vs Default Exports (CommonJS)

Default export uses module.exports = value. Named exports use module.exports = { fn1, fn2 }.

```
// Default
module.exports = function multiply(a,b){ return a*b; }

// Named
```

```
module.exports = {
  add: (a,b)=>a+b,
  sub: (a,b)=>a-b
};

// Import
const mul = require("./math");
const { add, sub } = require("./math");
```

Request and Response

Request (req) is what client sends: method, URL, headers, body. Response (res) is what server sends back: status code, headers, body.

```
// Express example
const express = require("express");
const app = express();
app.use(express.json());

app.get("/hello", (req, res) => res.send("Hello World"));

app.post("/login", (req, res) => {
  const { username, password } = req.body;
  if (username==="alisha" && password==="1234")
    res.status(200).json({message:"Login successful"});
  else
    res.status(401).json({message:"Unauthorized"});
});

app.listen(3000);
```